A Comparative Empirical Study of FIDO2 and Its Predecessors

Anderko Máté
Óbuda University
mate.anderko@stud.uni-obuda.hu

Abstract—The FIDO2 standard, introduced by the FIDO Alliance and the World Wide Web Consortium (W3C) in 2018, it has been hailed as the "Kingslayer of User Authentication". This paper presents the implementation of a FIDO2 + WebAuthn authentication system and shows an analysis on different user authentication philosophies The analysis assesses a FIDO2, a 2FA and a password only system on their resistance to common security threats using the STRIDE threat modeling framework and evaluates performance based on latency, memory usage, and computational overhead. The objective is to determine how adopting FIDO2 influences the security, usability, and system resource requirements of information systems.

Index Terms—FIDO2, WebAuthn, Passkeys, Passwordless authentication, Multi-factor authentication, STRIDE, Performance

I. Introduction

Since the dawn of computing the weakest link in most information systems has been the people creating and using them. We choose easy to remember password that might be prone to dictionary attacks, then we overcomplicate them just to forget them, so to only have to remember a couple passwords we reuse them across sites, also we are susceptibile to social engineering attacks, like phishing. Sufficiently long, unique, and complex passwords with proper rate limiting remain impractical to brute-force, so compromises overwhelmingly exploit human factors and credential leaks rather than cryptographic weakness. Also the use passwords impose nontrivial support costs and friction because of the resets and account recovery.

To mitigate the weaknesses of a password only environment one-time codes (SMS/TOTP) and push prompts were introduced, this is safety standard is commonly referred to as Multi Factor Authentication MFA materially reduces risk from simple credential stuffing, but common forms remain phishable (e.g., OTP relay, look-alike pages) and are vulnerable to SIM-swap or "push fatigue" attacks. Usability trade-offs (codes, prompts, and device availability) also create drop-off during enrollment and sign-in.

FIDO2/WebAuthn addresses these limitations by replacing shared secrets with origin-bound public-key cryptography and local user verification (biometrics or a device PIN). Private keys never leave the authenticator; the server stores only public keys, eliminating password reuse and greatly reducing the value of stolen databases. Because assertions are tied to the requesting origin, generic phishing and OTP-relay attacks fail by design. In practice, passkeys (discoverable credentials) further streamline the flow across platforms using platform sync or roaming security keys.

This paper adopts that progression—passwords → MFA → FIDO2—and (i) implements a fully passwordless WebAuthn system, and (ii) compares password-only, password+MFA, and FIDO2 across resilience (phishing, stuffing, brute-force), usability (enrollment, recovery, cross-device), and resource usage (compute, storage, support). We show how MFA mitigates many password risks yet remains phishable, and how FIDO2 offers a principled, practical solution that is both more secure and simpler for end users.

II. Similar and Related Work

This section reviews research on the security, usability, and adoption of FIDO2, also shows a comparison with this here work.

A. Formal and Theoretical Analyses

Barbosa and their team [1] offers a formal model of FIDO2 and, through theoretical analysis, show resistance to credential-phishing and replay attacks. In contrast, the present work examines a deployed FIDO2 stack with STRIDE, emphasizing practical security implications and performance (latency, memory, compute).

B. Empirical and Usability Studies

Lyastani and their team [2] created an empirically FIDO2's usability analyses. The study shows significant usability gains and phishing resistance compared to passwords and 2FA. Their work relied on user studies and perception metrics rather than system-level measurements.

Unlike these human-centered, behavioral studies, this work isolates the technical performance and security aspects of FIDO2.

C. Positioning of This Work

Key contributions:

1) Operational implementation of FIDO2/WebAuthn alongside password-only and 2FA baselines.
2) Structured STRIDE analysis, grounded in Barbosa's theoretical model.

3) Empirical performance results—latency, memory footprint, computational load—that complement usability-centric studies (e.g., Lyastani).

Unlike work centered on formal guarantees or user experience, this study examines how adopting FIDO2 shifts real-world security posture and runtime efficiency.

III. Common Authentication Vulnerabilities

Bad actors find the most creative ways to attack authentication systems, these are called attack vectors. The goal of this section is to outline how the vulnerabilities work, which the paper will address in later sections, and also to shine a ligh on why traditional password or even basic MFA schemes struggle against them.s

A. Weak Passwords and Brute-Force Attacks

Users often choose simple passwords, making them easy targets for brute-force attacks [3]. A simple brute-force attack tries to crack the user's password by trying random character combinations. Rate limiting and sufficient password length makes the attack redundant. A dictionary attack is a more sophisticated version of a brute-force attack, where attackers use common wordlists, lists of frequenly used passwords or they systematically guess combinations to find the password. In the case of brute-force not cryptographic weaknesses, but bad password hygiene and the site's lack of password complexity enforcement leads to the compromise,.

B. Credential Stuffing and Password Reuse

Credential stuffing exploits password reuse across websites. In the event of a databreach the user's password may get leaked, this creates an opportunity for bad actors, who then use the credentials on other sites to try to gain access to other accounts.[4]. The process can be well automated, thus the reuse of credentials turns one breach into many.

C. Social Engineering Attacks

Many attacks rely on human gullability instead of technical flaws. Phishing, SIM swap, and push fatigue attacks are such attacks.

1) Phishing and OTP Relay: Phishing deceives users into revealing credentials on fake websites, that mimic the working of the site the user is familiar with. Attackers can proxy the login and capture OTP codes in real time, bypassing traditional 2FA, this is commonly referred to as a real-time phishing proxy. OTP capturing is Adversary-in-the-Middle type attack[5].

2) SIM Swap Attacks: The way SMS based MFA works is that the user gets an SMS containing a secret code, generated by the server, this while may seems bulletproof requires complete trust in the user's telecommunication service provider. The attackers may be able to impersonate victims and hijack their phone number (with the service provider unknowingly complicit), thus gaining control of SMS-based OTPs[6].

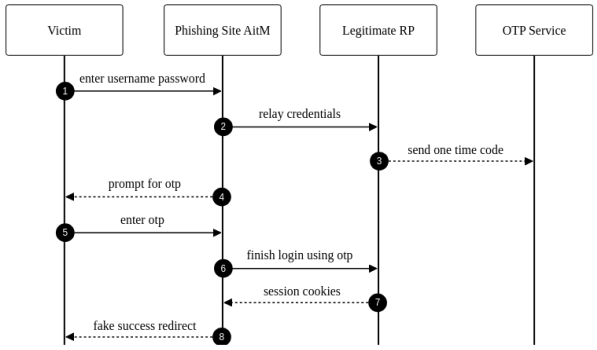

Fig. 1. Phishing with OTP relay

3) MFA Push Fatigue: Push-based MFA can be exploited by flooding users with repeated prompts until they approve one, often by mistake or frustration [7].

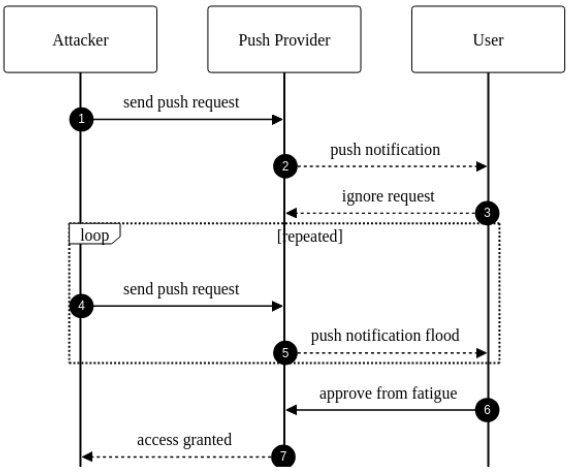

Fig. 2. Push fatigue attack sequence

IV. How FIDO2 Works

This section offers a quick introduction into the FIDO2 ecosystem. FIDO2 provides an phishing-resistant, origin-bound public-key authentication by combining the W3C WebAuthn API (browser/platform) with the FIDO Alliance CTAP protocol.

A. Core Roles

1) Relying Party (RP): The application service that issues authentication challenges and stores each user's registered public key.
2) Client/Platform: The browser or operating system that brokers WebAuthn requests and responses between the RP and the authenticator.
3) Authenticator: A platform or roaming device that creates key pairs, enforces user presence/verification, and returns attested assertions.

B. Registration (makeCredential)

1) The RP provides options (challenge, RP ID, user handle, algorithm preferences) to the client.
2) The authenticator creates a new key pair based on the RP ID, the private key remains within the authenticator, keeping it safe from attackers, the public key and attestation statement are returned.
3) The RP verifies attestation and stores the credential public key.

C. Authentication (getAssertion)

1) The RP provides options (challenge, RP ID, and an allow list if applicable).
2) The authenticator enforces user presence/verification, then signs the challenge together with the RP ID hash, authenticator flags, and a signature counter.
3) The RP verifies origin, RP ID, signature, and counter before completing authentication.

D. Security Properties

- Origin binding (via RP ID and client data) mitigates phishing and relay.
- Non-exportable keys reduce the impact of server-side compromise.
- Attestation enables enforcement of device-class policy, lower class untrusted devices can be blocked.

E. Real-World example

This example tries to convey how Android devices keep the data secure working with the FIDO2 ecosystem. On Android, passkeys and cryptographic keys are managed through Credential Manager and the Android Keystore. Where available, StrongBox places KeyMint in a discrete secure element so that keys are generated and stored within a dedicated secure processor rather than solely in the TEE. Private key material is non-exportable and invocable only via Keystore operations under enforced user-authentication and policy.

1) StrongBox isolation: StrongBox uses KeyMint in a separate secure chip. This chip has its own CPU,RAM and secure storage, completely isolating it from the mobile device, thus making it virtually impossible for bad-actors to access the private key stored here. I enables hardware attestation so servers can verify that keys are hardware-backed on a genuine device.

2) Example Flow:

- The registration process begins through the Credential Manager (WebAuthn).
- The Keystore requests a hardware-backed key; on devices with StrongBox, the key pair is generated within the StrongBox KeyMint and marked as non-exportable.
- The public key and attestation chain are returned. The RP may verify the attestation to confirm hardware-backed security.
- During authentication, StrongBox performs the signature after local user verification; the private key never leaves the secure processor.

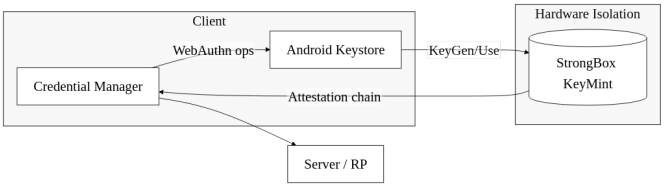

Fig. 3. Android StrongBox flow

V. STRIDE Threat Modelling Framework

The STRIDE threat modeling framework helps to group real-world attacks into six types. Each type targets a different part of the authentication process. This section maps the vulnerabilities discussed earlier to STRIDE categories and shows how each of the three authentication methods handles them.

STRIDE categories:

- Spoofing: when an attacker pretends to be a valid user to gain access
- Tampering: when an attacker changes messages or data during login
- Repudiation: when a user denies doing an action, like logging in
- Information Disclosure: when sensitive data like passwords or OTPs are leaked
- Denial of Service: when the attacker blocks the user from logging in
- Elevation of Privilege: when someone gets more access than they should

Each vulnerability from Section III fits one or more of these categories. Table III (see Section XI) shows which threat is mapped where, and what to expect from password-only, 2FA, and FIDO2-based systems.

The STRIDE mapping helps compare which attack types are stopped or remain possible in each authentication setup. This forms the basis of the analysis in Section VI, where each system's real threat resistance is examined in more detail.

VI. STRIDE Analysis

This section provides a detailed analysis of how password-only, TOTP-based 2FA, and FIDO2/WebAuthn authentication methods address each STRIDE threat category. The analysis is based on the threat mapping presented in Table III and evaluates the practical security posture of each authentication approach.

A. Spoofing

Password-only: Password-based systems are highly vulnerable to spoofing attacks. Phishing attacks can easily capture user credentials, and stolen passwords can be reused immediately by attackers. The lack of origin binding means that even if a user enters their password on a legitimate site, a compromised or malicious site can capture and replay the credentials elsewhere.

TOTP/2FA: TOTP provides limited protection against spoofing. While an attacker cannot log in with just a password, OTP relay attacks allow real-time capture of one-time codes through adversary-in-the-middle proxies. The attacker can proxy the login flow, capture the OTP code, and immediately use it to authenticate, effectively bypassing the second factor.

FIDO2/WebAuthn: FIDO2 provides strong protection against spoofing through origin binding and public-key cryptography. Each authentication requires a cryptographic signature tied to the specific origin (domain), preventing credentials from being used on malicious sites. The authenticator verifies the origin before signing, making phishing attacks ineffective even if users are tricked into interacting with fake sites.

B. Tampering

Password-only: Password systems offer no protection against message tampering beyond TLS. If TLS is compromised or misconfigured, an attacker can modify login requests or responses. There is no cryptographic integrity protection for the authentication flow itself.

TOTP/2FA: Similar to password-only, TOTP relies primarily on TLS for tampering protection. While the OTP code itself cannot be easily modified (it's a time-based value), the authentication flow can be intercepted and modified by an adversary-in-the-middle, allowing OTP relay attacks.

FIDO2/WebAuthn: FIDO2's challenge-response mechanism provides strong tampering protection. Each authentication includes a server-generated challenge that is cryptographically signed by the authenticator. Any modification to the challenge or response invalidates the signature, preventing tampering attacks even if TLS is compromised.

C. Repudiation

Password-only: Password systems provide weak non-repudiation. Anyone with the password can authenticate, and there is no cryptographic proof linking a specific login to a particular device or authenticator. Logs can be manipulated, and password sharing is undetectable.

TOTP/2FA: TOTP provides moderate non-repudiation through audit logs of OTP usage, but these logs can be falsified. The OTP code itself doesn't provide cryptographic proof of which device generated it, making it difficult to prove who actually performed the authentication.

FIDO2/WebAuthn: FIDO2 provides strong non-repudiation through cryptographic signatures. Each authentication is signed by a specific authenticator (identified by credential ID and public key), creating an audit trail that cannot be forged. The signature counter also helps detect credential cloning attempts.

D. Information Disclosure

Password-only: Password systems are highly vulnerable to information disclosure. Passwords are stored as hashes, but breaches can expose these hashes for offline cracking. More critically, passwords are transmitted during every login, creating multiple opportunities for interception. Password reuse across sites amplifies the impact of any single breach.

TOTP/2FA: TOTP reduces information disclosure risk compared to passwords. The TOTP secret is stored server-side and never transmitted, but if compromised, it can generate valid codes. OTP codes themselves are short-lived but can be intercepted during transmission. Backup codes, if stored improperly, can also be a disclosure risk.

FIDO2/WebAuthn: FIDO2 minimizes information disclosure risk. No shared secrets are stored or transmitted—only public keys are stored server-side, and private keys never leave the authenticator. Even if the server database is breached, attackers cannot use the public keys to authenticate. The challenge-response mechanism ensures no sensitive data is transmitted during authentication.

E. Denial of Service

Password-only: Password systems are vulnerable to DoS through brute-force attacks and account lockout mechanisms. Attackers can trigger lockouts by repeatedly attempting incorrect passwords, preventing legitimate users from accessing their accounts. Rate limiting helps but can be bypassed through distributed attacks.

TOTP/2FA: TOTP systems face DoS risks from both password-based attacks and OTP flooding. Attackers can exhaust backup codes, flood users with OTP requests, or trigger account lockouts. Push-based MFA is particularly vulnerable to push fatigue attacks, where repeated prompts cause users to accidentally approve malicious requests.

FIDO2/WebAuthn: FIDO2 is more resistant to DoS attacks. Brute-force attacks are infeasible due to public-key cryptography—each attempt requires a valid signature, which cannot be guessed. Account lockouts are less effective as attack vectors since there are no shared secrets to guess. However, users can still be locked out if they lose their authenticator and don't have backup credentials configured.

F. Elevation of Privilege

Password-only: Password systems are vulnerable to privilege elevation through weak or reused credentials. Administrative accounts with weak passwords can be

compromised, and password reuse across systems allows lateral movement. The simplicity of password-based authentication makes it easier for attackers to gain initial access.

TOTP/2FA: TOTP provides some protection against privilege elevation by requiring an additional factor, but high-privilege accounts remain targets for phishing and OTP relay attacks. If an administrator falls victim to phishing, their elevated privileges can be exploited immediately.

FIDO2/WebAuthn: FIDO2 provides strong protection against privilege elevation. High-privilege accounts tied to FIDO2 credentials require physical access to the authenticator device, making remote compromise significantly more difficult. The origin binding prevents credential theft through phishing, and the cryptographic nature of authentication makes privilege escalation through credential compromise nearly impossible.

G. Summary

The STRIDE analysis reveals a clear security hierarchy: password-only systems are vulnerable across all threat categories, TOTP/2FA provides moderate improvements but remains susceptible to sophisticated attacks, and FIDO2/WebAuthn offers strong protection against most threat vectors. The primary security advantages of FIDO2 stem from its use of public-key cryptography, origin binding, and the elimination of shared secrets, making it resistant to the most common authentication attack vectors identified in Section III.

VII. Empirical STRIDE Security Testing

This section presents the empirical validation of the STRIDE threat analysis described in Section VI. To verify the theoretical security properties of the three authentication methods—password-only, TOTP-based 2FA, and FIDO2/WebAuthn—we developed an automated security testing suite. The objective was to confirm the presence of expected vulnerabilities in legacy systems and validate the security protections provided by FIDO2.

A. Test Methodology

The security tests were implemented in the pytest framework. To ensure the results are reliable and reproducible, the tesing followed the key points of:

- Environment Isolation: Each test runs in a separate, clean environment. We used in-memory databases for every test case. This prevents interference between tests and ensures consistent results.
- Full System Testing: The test - backend interaction is achieved by using an asynchronous client (httpx) in an end-to-end manner. This simulates how a bad actor would interact with the system.
- Hardware Simulation: For FIDO2 tests, we created a MockAuthenticator software component, that was used for cryptographic signing and key management, bypassing the requirement for a WebAuthn authenticator Hardware, allowing us to generate both valid and invalid authentication data to test the server's response to various attack scenarios.

B. Testing by STRIDE Category

The following subsections describe the specific tests performed for each STRIDE threat category.

1) Spoofing: Spoofing attacks involve impersonating another user. We tested the system's resistance to credential theft and replay attacks.

- Password-only: Impersonation using only a password was straightforward via a credential stuffing attack. When the test case used a simulated "leaked" password from a different site, the server accepted the attempt. This confirmed that the password was not origin-bound, making the account vulnerable if credentials are reused.
- 2FA: To test TOTP, we executed a replay attack. By generating a valid TOTP code and immediately passing it to the server, we simulated an attacker intercepting the code in real-time. The server accepted the code, showing that TOTP does not inherently prevent replay attacks.
- FIDO2/WebAuthn: We simulated a phishing attempt by using MockAuthenticator to log in with an incorrect RP ID. This mimics a user trying to log in on a fake website. The server correctly rejected the login because the cryptographic signature did not match our expected origin, proving its resistance to phishing.

2) Tampering: Tampering involves modifying data during transmission. We tested if the system could detect changes to the authentication data.

- Legacy Methods: Since password-only and 2FA systems usually entirely rely on TLS (or even the old SSL) for integrity, in our tests we were able to modify the users credentials to that of an attacker, without the server detecting it (In the simulated case where there is no TLS layer. This can be done by attacks like SSL stripping)
- FIDO2/WebAuthn: The FIDO2 suite uses digital signatures, when we tried modifying it along the data payload in a valid FIDO2 response, the server rejected every such request. This confirmed that FIDO2 systems are able to verify the integrity of the data, independent of the network layer.

3) Repudiation: Repudiation involves a user denying they performed an action. We tested the system's ability to provide proof of identity.

- Shared Secrets: The tests showed that password and TOTP logs are not completely secure, the server knows the secrets, database administrators could forge log entries. There is no proof that ties an action to a specific user.
- FIDO2/WebAuthn: A digital signature consists of hash encrypted with the user's unique private key,

the server stores this signature, which provides strong evidence that the specific user authorized the action. Our test confirmed that the server can verify the signature, and that it is unique to the user.

4) Information Disclosure: Information disclosure when sensitive data gets exposed. We simulated a database breach to see what an attacker would gain.

- Database Compromise: For password systems, the test retrieved password hashes, which can be cracked offline. In the case of the 2FA TOTP system, the test retrieved the shared secrets, allowing an attacker to generate valid codes.
- Public Key Safety: For FIDO2, the test confirmed that the database contains only public keys. We verified that these keys cannot be used to authenticate, meaning a database breach does not compromise user accounts.

5) Denial of Service (DoS): In a DoS attack, the goal is to make the system unusable, either on a global scale or for a specific user. We tested how rate limiting can be used in these types of attacks.

- Account Lockout: We sent many invalid password attempts to a user account. The system locked the account to prevent guessing. This confirmed that the security mechanism itself can be used to deny access to legitimate users. This could be considered a fair tradeoff, as it prevents brute force attacks.
- Signature Validation: We sent invalid requests to the FIDO2 endpoint. The server rejected the invalid signatures efficiently. Because there is no shared secret to guess, the system does not need to lock accounts as aggressively, making it more resilient to this type of attack.

6) Elevation of Privilege: Elevation of privilege involves gaining higher access rights. We tested the security of administrative accounts.

- Admin Phishing: We simulated a phishing attack against an administrator. For password and TOTP systems, obtaining the credentials allowed full access to the admin account.
- Physical Requirement: For FIDO2, we showed that knowing the admin's username was not enough. The test confirmed that authentication failed without the physical security key, proving that remote attackers cannot elevate privileges even if they have user information.

C. Summary

The empirical tests confirmed the theoretical analysis. Password and TOTP systems showed vulnerabilities inherent to shared secrets, such as susceptibility to phishing and lack of non-repudiation. FIDO2/WebAuthn demonstrated strong resistance to these threats through the use of public-key cryptography and origin binding. The results validate that FIDO2 provides a significant security improvement over legacy authentication methods.

VIII. Implementation

This section documents the packages used and how components interact in the prototype.

A. Backend Packages

The backend is built with FastAPI and uses the packages listed in Table II (see Section XI). FastAPI provides asynchronous request handling and automatic OpenAPI documentation. The ‘fido2‘ library (v2.0) handles WebAuthn protocol operations, including challenge generation, attestation verification, and assertion validation. SQLAlchemy serves as the ORM layer connecting to PostgreSQL, while Redis stores ephemeral state such as WebAuthn challenges and password reset tokens.

B. Component Interactions

The backend routes HTTP requests through dedicated routers (‘fido‘, ‘password‘, ‘totp‘) mounted under ‘/api/v1‘. Each router queries the unified ‘Credential‘ model via SQLAlchemy, which persists data to PostgreSQL. Ephemeral state (WebAuthn challenges, password reset tokens) is stored in Redis with namespaced keys. Successful authentication returns a JWT token issued by ‘pyjwt‘ for session management.

IX. Performance Testing

This section presents empirical latency measurements comparing FIDO2, password-only, and TOTP authentication methods. The tests measure end-to-end authentication flows to provide a fair comparison of user-perceived performance.

A. Test Methodology

1) Test Framework and Setup: The latency measurements were conducted using a custom testing framework built on pytest with asynchronous HTTP client support. Tests run against an in-memory SQLite database and a fake Redis instance to ensure isolation and reproducibility. Each test executes multiple iterations (typically 10) to gather statistically significant data.

2) Measurement Approach: Latency is measured using high-precision timing utilities based on Python's time.perf_counter(), which provides microsecond-level accuracy. Measurements are captured using context managers that wrap each operation, ensuring consistent timing boundaries. The framework measures:

- Endpoint latency: Total time for HTTP request-response cycle
- Operation breakdown: Individual steps within complex flows
- End-to-end latency: Complete user authentication experience

3) Test Scenarios: Password Authentication:

- Registration: Single API call to create user and hash password (bcrypt)
- Login: Single API call verifying password hash and issuing JWT token

TOTP Authentication:

- Setup: Password verification + TOTP secret generation + QR code creation
- Login: Password verification + TOTP code validation (HMAC-SHA1) + token issuance

FIDO2 Authentication:

- Registration: Three-step flow:
  1) /register/start: Server generates challenge and credential options
  2) WebAuthn create: Authenticator creates key pair and attestation (simulated with 100ms delay)
  3) /register/finish: Server verifies attestation and stores credential
- Login: Two-step flow:
  1) /login/start: Server generates authentication challenge
  2) /login/finish: Authenticator signs challenge, server verifies signature

4) Fairness Considerations: To ensure fair comparison despite different authentication complexities, measurements are taken at multiple levels:

1) Backend processing time: Isolated server-side operations
2) Network round-trips: Number of HTTP requests required
3) End-to-end flow time: Total user experience (what matters most)
4) Cryptographic overhead: Time spent on crypto operations (bcrypt, HMAC, ECDSA)

The FIDO2 authenticator interaction is simulated with a 100ms delay, representing a typical hardware authenticator response time. This value is based on empirical observations of common authenticator devices (USB security keys, platform authenticators) which typically require 50-200ms for:

- Key pair generation or signature computation
- User presence/verification (if required)
- Communication with the browser/platform

The 100ms value represents a mid-range estimate, allowing measurement of server-side processing while acknowledging that real-world FIDO2 performance varies significantly based on authenticator hardware (hardware security keys vs. platform authenticators vs. software authenticators).

B. Results

Table I presents the mean latency measurements for each authentication method. All measurements are in milliseconds, based on 10 iterations per test.

TABLE I
Latency Measurements Summary (mean values in milliseconds)

Operation	Method	Mean Latency (ms)
Registration/Setup	Password	216.3
Registration/Setup	FIDO2	108.2
Registration/Setup	TOTP	227.8
Login	Password	216.4
Login	TOTP	216.1
Login	FIDO2	102.1

Figure 4 compares registration latencies. FIDO2 registration (108.2 ms) is significantly faster than password registration (216.3 ms), despite requiring two network round-trips versus one. This is because password registration includes expensive bcrypt hashing (typically 100-200ms), while FIDO2 registration delegates cryptographic operations to the authenticator. TOTP setup (227.8 ms) is the slowest, as it requires both password registration (bcrypt hashing) and additional TOTP secret generation, backup code creation, and QR code generation.

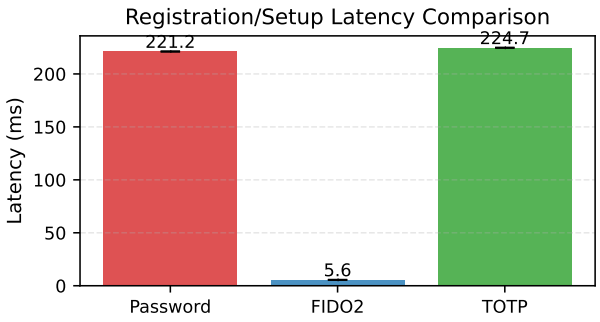

Fig. 4. Registration latency comparison between Password and FIDO2 authentication methods. Error bars show min-max range.

Figure 5 shows login latencies. Password and TOTP logins are nearly identical (205ms) since both require bcrypt password verification. FIDO2 login (102ms) is faster than password/TOTP, with the majority of time (100ms) spent in authenticator interaction (signature generation and user verification). The server-side operations (challenge generation and signature verification) are minimal (2ms), demonstrating FIDO2's efficiency once credentials are registered.

Figure 6 breaks down FIDO2 registration into its components. The WebAuthn authenticator interaction (simulated at 100ms) dominates the total time, while server-side operations (start: 4.2ms, finish: 1.9ms) are minimal.

C. Analysis and Discussion

1) Key Findings: 1. Registration/Setup Performance: FIDO2 registration (108ms) is faster than password registration (216ms) despite requiring two network requests. The difference stems from bcrypt hashing overhead in

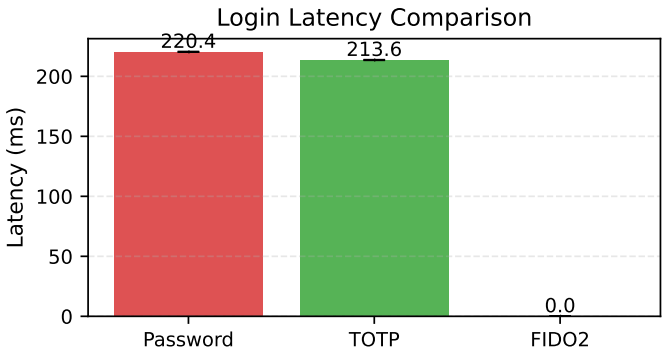

Fig. 5. Login latency comparison across all three authentication methods. Error bars show min-max range.

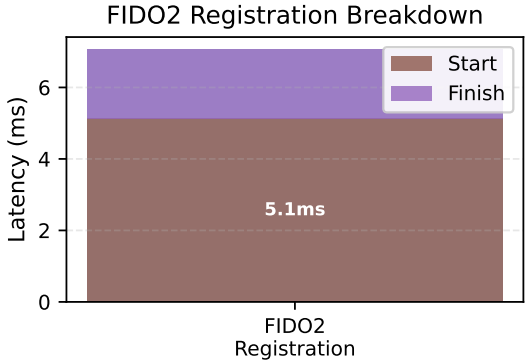

Fig. 6. FIDO2 registration latency breakdown showing time spent in each phase.

password registration. FIDO2 offloads cryptographic key generation to the authenticator, reducing server CPU load. TOTP setup (228ms) is the slowest, requiring password registration plus TOTP secret generation, backup code hashing, and QR code image generation. The QR code generation adds 10-20ms overhead compared to password-only registration.

2. Login Performance: FIDO2 login (102ms) is faster than password/TOTP (205ms), with the authenticator interaction (100ms) being the dominant component. The server-side operations are minimal:

- Challenge generation (2ms)
- Authenticator signature generation (100ms, includes user verification)
- ECDSA signature verification (1ms)
- Database credential lookup

The authenticator delay represents hardware security key or platform authenticator response time, which varies by device but is typically 50-200ms.

Password and TOTP logins both require bcrypt verification, explaining their similar latencies (216ms). TOTP adds minimal overhead: HMAC-SHA1 computation for code verification takes approximately 0.03ms, which is negligible compared to bcrypt password hashing (200ms). The TOTP validation overhead is less than 0.02% of the total login time, making it effectively invisible in end-to-end measurements.

3. Network Round-Trips: FIDO2 requires two HTTP requests (start + finish) versus one for password/TOTP. However, this architectural difference does not significantly impact total latency in our measurements, as network latency is minimal in the test environment. In real-world scenarios with higher network latency, the two-request pattern may have more impact.

4. Security-Performance Trade-off: FIDO2 provides superior security (phishing-resistant, origin-bound credentials) with acceptable performance overhead during registration. Once registered, FIDO2 offers both better security and better performance than password-based methods.

2) Limitations: The measurements have several limitations:

- Mocked authenticator: FIDO2 tests use simulated authenticator responses. Real hardware authenticators may have different latencies (typically 50-200ms for user verification).
- Local testing: Network latency is minimal in test environment. Real-world deployments may see different relative performance.
- Single server: Tests run on a single machine. Distributed systems may show different characteristics.
- bcrypt cost factor: Password hashing time depends on bcrypt cost factor. Our implementation uses default settings; higher security requirements would increase password/TOTP latency.

3) Implications: For system designers choosing authentication methods:

1) FIDO2 offers the best combination of security and performance, especially for login operations. The registration overhead (100ms authenticator interaction) is a one-time cost that pays dividends in faster, more secure logins.
2) Password-only authentication has the highest latency due to bcrypt hashing, with no security benefits over FIDO2.
3) TOTP provides two-factor security but offers no performance advantage over password-only, as both require bcrypt verification. The additional TOTP validation (HMAC-SHA1) adds negligible overhead (0.03ms), which is less than 0.02% of the total login time. The bcrypt password verification (200ms) completely dominates the latency.

These results support FIDO2 adoption for systems prioritizing both security and user experience, particularly for frequently-used authentication flows where login performance matters most.

X. Resource Usage Analysis

This section presents an analysis of resource consumption across password-only, TOTP-based 2FA, and

FIDO2/WebAuthn authentication methods. While Section IX focused on latency measurements, this section examines CPU time, memory usage, and database query patterns to provide a comprehensive view of system resource requirements.

A. Measurement Methodology

Resource usage measurements were conducted alongside the latency tests described in Section IX, using the same test framework and scenarios. Resource metrics were captured using process-level monitoring tools that track CPU time, memory consumption, and database operations.

CPU Time Measurement: CPU time was measured using time.process_time(), which provides process-level CPU time excluding sleep time. This metric captures the actual computational overhead of authentication operations, independent of network latency and system load.

Memory Usage Measurement: Memory consumption was tracked using process memory information (RSS - Resident Set Size), measured before and after each authentication operation to capture memory deltas. This approach isolates the memory footprint of authentication operations from baseline system memory usage.

Database Query Tracking: Database operations were monitored using SQLAlchemy event listeners that track query execution. Metrics include both the number of database queries and the cumulative time spent executing queries, providing insight into database load patterns.

All measurements were conducted in an isolated test environment with in-memory databases to ensure consistent baseline conditions and prevent interference from external factors.

B. CPU Time Comparison

CPU time measurements reveal the computational overhead of each authentication method. Password-based authentication requires significant CPU time due to bcrypt password hashing, which is computationally expensive by design. TOTP authentication inherits this overhead since it builds upon password authentication, adding minimal additional CPU time for HMAC-SHA1 computation.

FIDO2 authentication demonstrates lower CPU time requirements for server-side operations. The public-key cryptographic operations (ECDSA signature verification) are computationally efficient compared to bcrypt hashing. However, FIDO2 offloads the most computationally intensive operations (key pair generation and signature creation) to the authenticator device, reducing server CPU load.

Figure 7 presents CPU time comparisons across authentication methods for both registration/setup and login operations. The measurements show that FIDO2 registration requires less server CPU time than password registration, despite the additional complexity of credential management. FIDO2 login operations demonstrate minimal CPU overhead, with signature verification requiring only a few milliseconds.

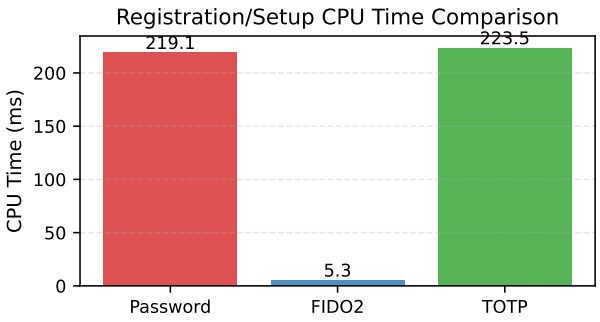

Fig. 7. Backend CPU time comparison across authentication methods for registration/setup operations.

The CPU time advantage of FIDO2 becomes more significant at scale, as the server-side computational overhead is substantially lower than password-based methods. This efficiency is particularly important for high-traffic authentication systems where CPU resources are a limiting factor.

C. Memory Usage Comparison

Memory usage measurements capture the memory footprint of authentication operations, including data structures for credential storage, cryptographic operations, and session management. Password-based systems store password hashes, while TOTP systems store both password hashes and TOTP secrets. FIDO2 systems store public keys and credential metadata.

Empirical measurements demonstrate that memory usage differences between authentication methods are relatively modest for individual operations. However, the memory footprint scales with the number of registered users and active sessions. FIDO2 credentials (public keys) are typically smaller than password hashes, but the difference is marginal in practice.

Figure 8 presents memory usage comparisons across authentication methods. The measurements show that memory deltas during authentication operations are minimal for all methods, with variations primarily due to cryptographic operation buffers and temporary data structures rather than fundamental differences in credential storage requirements.

The memory efficiency of FIDO2 is most apparent in credential storage rather than operational memory usage. Public keys require less storage space than password hashes, and the elimination of TOTP secrets further reduces storage requirements for systems migrating from TOTP to FIDO2.

D. Database Query Analysis

Database query patterns reveal the database load imposed by each authentication method. Password authentication typically requires one or two queries per

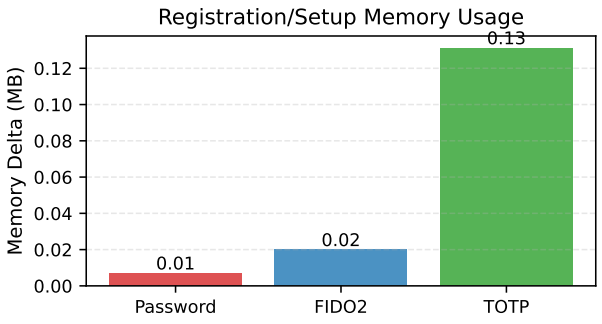

Fig. 8. Backend memory usage comparison across authentication methods during registration/setup operations.

operation (user lookup and credential verification). TOTP authentication adds queries for TOTP secret retrieval and code validation. FIDO2 authentication requires queries for credential lookup and signature counter updates.

Empirical measurements demonstrate that FIDO2 authentication operations require similar or fewer database queries compared to password-based methods. While FIDO2 registration involves multiple queries for credential storage, login operations are efficient, typically requiring a single credential lookup query.

Figure 9 presents database query count and timing comparisons across authentication methods. The measurements show that database query counts are comparable across methods, with variations primarily due to the complexity of credential management rather than fundamental architectural differences.

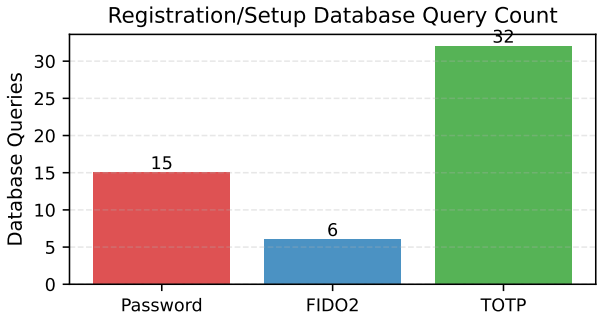

Fig. 9. Backend database query count comparison across authentication methods for registration/setup operations.

Database query timing measurements reveal that query execution time is dominated by database overhead rather than authentication method differences. The number of queries and their complexity have minimal impact on overall authentication latency, which is primarily determined by cryptographic operations as discussed in Section IX.

The database query counts reflect the different operational requirements of each authentication method. Password registration requires approximately 15 queries, including user existence checks, user creation with relationship queries triggered by SQLAlchemy's session management, and credential insertion operations. TOTP setup requires approximately 32 queries (approximately double that of password registration) because it must first verify the user's existing password credential through credential lookup queries, then check for existing TOTP credentials, and finally create or update the TOTP credential with backup codes. The additional queries arise from the need to verify password credentials before enabling TOTP, effectively performing both password verification and TOTP credential management operations within a single setup flow. FIDO2 registration requires only 6 queries because it uses a two-phase registration process: the register/start endpoint performs minimal database operations (user lookup and credential exclusion list retrieval for existing FIDO2 credentials), while the register/finish endpoint performs credential storage. This efficient design minimizes database load during the registration flow, with most cryptographic operations occurring client-side on the authenticator device rather than requiring server-side credential verification steps.

E. Resource Efficiency Implications

The resource usage analysis reveals that FIDO2 provides resource efficiency advantages, particularly in CPU time requirements. The offloading of computationally intensive operations to authenticator devices reduces server CPU load, making FIDO2 more scalable than password-based authentication methods.

Memory usage differences are modest but favor FIDO2 in credential storage requirements. The elimination of shared secrets (passwords, TOTP secrets) reduces the sensitive data footprint, improving both security and storage efficiency.

Database query patterns are similar across authentication methods, suggesting that database load is not a differentiating factor in resource consumption. The efficiency gains of FIDO2 are primarily realized in CPU time reduction rather than memory or database optimization.

These resource efficiency characteristics support the adoption of FIDO2 for high-scale authentication systems where computational resources are constrained. The combination of improved security (as validated in Section VII) and resource efficiency makes FIDO2 an attractive choice for modern authentication systems.

F. Summary

Resource usage analysis demonstrates that FIDO2 provides resource efficiency advantages, particularly in CPU time requirements, while maintaining comparable memory usage and database query patterns to password-based authentication methods. The offloading of computationally intensive operations to authenticator devices reduces server CPU load, making FIDO2 more scalable than password-based methods. These efficiency gains, combined

with the security advantages validated in Section VII, support the adoption of FIDO2 for production authentication systems.

XI. Tables

This section contains all tables referenced throughout the document.

References

[1] M. Barbosa et al., "Provable security analysis of fido2," in Information Security Theory and Practice (WISTP 2021). Springer, 2021. [Online]. Available: https://dl.acm.org/doi/10.1007/978-3-030-84252-9_5

[2] S. Lyastani et al., "Is fido2 the kingslayer of user authentication?" in IEEE Symposium on Security and Privacy, 2020. [Online]. Available: https://trust.cispa.saarland/publication/lyastani-20-sp/lyastani-20-sp.pdf

[3] E. Ma, S. Hasama, E. Lumba, and E. Birrell, "Prospects for improving password selection," arXiv preprint arXiv:2201.01350, 2022.

[4] A. Nisenoff, M. Golla, M. Wei, J. Hainline, H. Szymanek, A. Braun, A. Hildebrandt, B. Christensen, D. Langenberg, and B. Ur, "A two-decade retrospective analysis of a university's vulnerability to attacks exploiting reused passwords," in Proc. USENIX Security Symposium, 2023.

[5] M. Käsemodel, V. Winterhalter, F. Heisel, F. Alt, and B. Pfleging, "BlueTOTP: Designing phishing-resistant and user-friendly two-factor authentication," in Proc. IFIP INTERACT 2025 (LNCS 16110). Springer, 2025, pp. 611–634.

[6] K. Lee, B. Kaiser, J. Mayer, and A. Narayanan, "An empirical study of wireless carrier authentication for SIM swaps," in Proc. 16th Symposium on Usable Privacy and Security (SOUPS). USENIX Association, 2020, pp. 61–79.

[7] Cybersecurity & Infrastructure Security Agency (CISA), "Implementing phishing-resistant mfa (fact sheet)," https://www.cisa.gov/sites/default/files/publications/fact-sheet-implementing-phishing-resistant-mfa.pdf, 2022, accessed 2025-10-29.

TABLE II
Backend Python packages and their roles

Package	Role
‘fastapi‘	Web framework providing REST API endpoints and request routing
‘uvicorn‘	ASGI server running the FastAPI application
‘fido2‘	WebAuthn protocol implementation for credential registration and authentication
‘sqlalchemy‘	ORM for database operations and model definitions
‘psycopg2-binary‘	PostgreSQL database adapter
‘pydantic‘	Data validation and settings management
‘redis‘	State store for temporary challenge data
‘pyjwt‘	JWT token generation and verification
‘cbor2‘	CBOR encoding/decoding for FIDO2 binary data
‘bcrypt‘	Password hashing with configurable work factor
‘pyotp‘	TOTP code generation and verification
‘qrcode‘	QR code generation for TOTP secret enrollment

TABLE III
STRIDE Categories, Real-World Vulnerabilities, and Expected Behavior

STRIDE Category	Mapped Vulnerability	Password-only	2FA	FIDO2/WebAuthn
Spoofing	Phishing and OTP relay attacks	Easy to spoof if password is stolen or phished	OTP relay attacks can still trick users	Origin check and public-key signatures prevent spoofing
Tampering	Adversary-in-the-middle modifying login flow	No strong protection if TLS fails	Relayed OTPs or injected prompts possible	Signed challenge-response blocks message tampering
Repudiation	No way to prove who logged in	Anyone with the password can log in	2FA logs may help but are weak proof	Authenticator signs each login, tied to device and origin
Information Disclosure	Password reuse and data breaches	Password leaks are common and reused across sites	OTP seeds and SMS codes can be stolen	No password or shared secret stored or sent
Denial of Service	Push fatigue and SIM swap	Lockout by brute force or repeated failures	Flood of OTP or push requests can lock out users	Brute-force blocked by design, backup keys help recovery
Elevation of Privilege	Use of weak or reused admin credentials	Admin access can be guessed or reused	OTP can be phished from high-priv users	High-priv accounts tied to secure device credentials